Khanh Dong
CAP 5705
December 10th, 2025

Final Project Report

Overview

The goal of the project is to implement a 3D visualization of an object placed within an indoor environment using skybox, featuring real-time environment mapping and bump mapping using modern OpenGL. While skybox and environment mapping were successfully implemented, bump mapping could not be completed within the project timeframe.

Skybox

The indoor scene was downloaded for free from Polyhaven as an HDRI. I then converted it into a cubemap using this website. The skybox was implemented using a cube mapping technique where six individual texture images (representing the right, left, top, bottom, front, and back walls of an indoor room) were combined into a single cubemap texture. A unit cube was rendered around the camera position, with its vertices defined to cover the entire view frustum. In the rendering pipeline, the skybox is drawn after the main scene objects using a dedicated shader program that samples from the cubemap based on the view direction vector. To ensure the skybox appears at infinite distance, the translation component is removed from the view matrix before rendering, and depth testing is temporarily set to GL_LEQUAL so the skybox only fills background pixels not occupied by closer objects. The implementation was based on this tutorial.

The 3D object used for the scene is a chair obj file downloaded from Free3D. The chair object was rendered using a standard lighting shader that calculated Phong illumination based on vertex normals and a single light source position. The OBJ file contained faces defined as quads (four vertices per face), which needed to be converted to triangles during loading since OpenGL primarily renders triangles. The parser detected quad faces and split each into two triangles using a fan triangulation pattern. The rendering pipeline then transformed these triangular faces through model, view, and projection matrices, and in the fragment shader computed ambient, diffuse, and specular lighting components using the chair's material properties and light position. The chair's appearance was determined either by a diffuse texture (leather and wood) mapped using UV coordinates or by procedural vertex colors if no texture was available. The implementation is based on this tutorial.

Figure 1. Skybox scene where the cube maps are joined at the edge

Figure 2. Chair object with multiple textures

Environment Mapping

Environment mapping was implemented using cube mapping, in the fragment shader, reflection vectors are calculated using the formula R = reflect(I, N) where I is the incident vector from the fragment to the camera and N is the surface normal, with these vectors used to sample the

cubemap to obtain reflected colors that are blended with the chair's base material. The implementation is based on this tutorial.

Figure 3. The object with reflection of the surrounding cube maps

Bump Mapping

I first convert the texture images of the chair object to their normal map using this website. The bump mapping implementation is supposed to follows a multi-stage pipeline starting with vertex data preparation where tangents and bitangents are calculated from the mesh geometry to create a Tangent-Bitangent-Normal (TBN) matrix that transforms normals from tangent space to world space. In the vertex shader, this TBN matrix is constructed from the transformed vertex normals, tangents, and bitangents, while also passing texture coordinates and fragment positions to the fragment shader. The fragment shader then samples the normal map texture, converts its RGB values from the [0,1] range to normalized [-1,1] vectors in tangent space, and transforms these vectors using the TBN matrix to obtain world-space normals that represent surface details. These perturbed normals are then used in lighting calculations instead of the original vertex normals, creating the illusion of surface bumps and grooves without modifying the actual geometry, while the displacement mapping is handled separately through parallax occlusion mapping that adjusts texture coordinates based on height map samples to simulate depth.

The implementation encountered several critical issues, which I have theorized to be the followings. Memory leaks occurred from creating default textures every frame without cleanup, gradually consuming GPU memory until the system froze entirely. Shader compilation errors

that happen when texture samplers were not properly bound, causing only black renders. Performance degradation stemmed from excessive texture switching and redundant uniform location queries each frame. These issues made systematic testing and debugging exceptionally difficult, ultimately preventing completion within the project's timeframe.

Conclusions

Working on this project on my own, I have learned the practical challenges of integrating multiple advanced OpenGL techniques. Understanding depth management for creating a seamless environment in the skybox, and how to implement quads-to-triangle conversion for rendering obj files. With environment mapping, I understood view-dependent reflections, vector mathematics for reflection calculations, and real-time parameter blending. From the failed bump mapping attempt, I learned about tangent space transformations, and the critical importance of proper resource management. The project can be improved by fixing the bump mapping implementation with proper tangent calculations and memory management, while future work could extend to dynamic cubemap generation, PBR materials, or shadow mapping.